

Chapter 1

A ROS 2 Module for Analytic Model Learning

1.1 Overview

Deliverable D3.7 [1] introduced the Physics-Aware Modeling Module (PhysicsAMM) and its symbolic-regression engine ANSR, and concluded with the plan to expose the module as a ROS 2 node following the best practices of the CoreSense/ROS development guidelines D5.1 [2]. This chapter documents that module. It consists of two ROS 2 packages hosted in the `ros2/` folder of the ANSR repository:

- `physics_amm` — the wrapper node. It manages learning sessions, runs the unmodified ANSR core as a supervised subprocess, and serves the learned analytic models;
- `physics_amm_msgs` — the interface definitions (messages, services, and one action), packaged separately as recommended by D5.1 for new interfaces.

D3.7 defined three operation modes for the module. Each maps onto exactly one ROS 2 interface:

- *Training a new model* — the `learn_model` action: a long-running, cancellable learning run with periodic feedback;
- *Updating an existing model* — the `set_session_data` service: the training data of a running session is replaced on the fly and the search adapts to the new data without a restart;
- *Querying an existing model* — the `get_model` service: the current best-so-far analytic models are returned at any time, both during and after a learning run.

Learning and retrieval are deliberately decoupled: a learning run may take minutes to hours, while a query returns immediately with the best models found so far. The module targets ROS 2 Jazzy Jalisco on Ubuntu 24.04 [3], is released under the Apache-2.0 license, and ships with a parameterised launcher, a configuration file of node parameters, and a six-layer test suite (Section 1.8).

1.2 ANSR from an integration perspective

Four runtime characteristics of the ANSR core, described in the preceding chapters and in [4], shape the design of the ROS layer:

1. **Long-running, parallel computation.** A learning run is an asynchronous neuro-evolutionary search whose gradient-based coefficient tuning is distributed over a pool of worker processes (Dask [5], one PyTorch instance per worker). Runs last from minutes to hours, and the worker pool is managed dynamically over the course of a run.

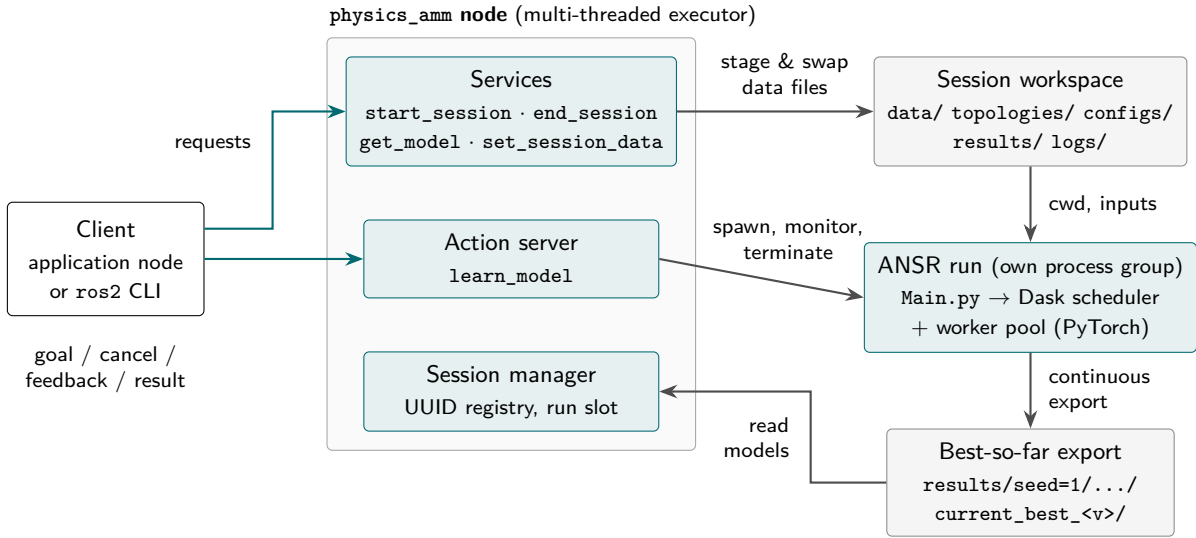


Figure 1.1: Architecture of the `physics_amm` module. Clients interact with the node through four services and one action. Each learning session owns a private workspace; the ANSR core runs as a supervised subprocess with the workspace as its working directory and continuously exports its best-so-far models, which the node reads to serve queries and to publish action feedback.

2. **File-based data interface.** Training and validation data are CSV files, and all inputs (data, master topology, optional constraints) are resolved relative to a structured working directory with dedicated subdirectories for each input kind.
3. **Runtime data updates.** The core monitors its training-data file while the search is running. When the file changes, the data are reloaded, an internal *dataset version* is advanced, and all fitness comparisons are made dataset-aware, so the search continues seamlessly on the new data.
4. **Continuous best-so-far export.** The evolving populations are observable: the archive of non-dominated solutions and the live search population are continuously exported to disk, one snapshot directory per population and dataset version, refreshed whenever the populations improve (with a configurable minimum interval between refreshes).

These characteristics are what make the three operation modes of Section 1.1 natural to provide: the file-based interface and runtime data monitoring directly support data exchange by file reference and on-the-fly updates, and the continuous export makes best-so-far queries possible without touching the running computation. The ROS layer builds on each of them.

1.3 Architecture

The module follows the session-based wrapper pattern established in the CoreSense ecosystem by `coresense_vampire` [6], which wraps another long-running computation (the Vampire theorem prover) behind a session/action interface. Figure 1.1 shows the data flow.

Sessions. All operations are scoped by a session: a client opens a session (`start_session`, returning a UUID), runs learning goals and queries against it, and closes it (`end_session`). Sessions isolate concurrent users from each other: each session has its own workspace and its own results directory.

Table 1.1: Public interface of the `physics_amm` node. All types are defined in `physics_amm_msgs`.

Name	Kind	Type	Purpose
<code>start_session</code>	service	<code>StartSession</code>	open a session; returns a UUID
<code>end_session</code>	service	<code>EndSession</code>	close a session, release its resources
<code>learn_model</code>	action	<code>LearnModel</code>	run learning: long-running, cancellable
<code>get_model</code>	service	<code>GetModel</code>	query best-so-far analytic models
<code>set_session_data</code>	service	<code>SetSessionData</code>	replace training data mid-run

Subprocess isolation. The ANSR core runs in its own Python process, launched and supervised by the node. This keeps the scientific computing stack (PyTorch, Dask, SymPy) fully independent of the ROS middleware process: the node itself performs no heavy computation in callbacks — in line with D5.1 — and the core’s worker pool is created in a clean interpreter dedicated to the run. The subprocess is placed in its own process group at launch, which lets the node manage the entire worker pool as a single unit throughout the run (Section 1.6).

Per-session workspaces. For each learning goal, the node stages the input files into a fresh workspace that reproduces the core’s directory conventions (`data/`, `topologies/`, `configs/`, `results/`) and starts the core with the workspace as its working directory. Inputs are passed to the core by bare filename, exactly as in a manual invocation. Staging copies the client’s files, so later in-place data updates never modify the originals.

Data by file reference. Training data cross the ROS interface as file paths rather than in-message payloads. This matches the core’s file-based data interface, keeps messages small, and makes the on-the-fly data update (Section 1.4) a natural operation: the node replaces the staged file, and the core picks up the change through its runtime data monitoring.

Minimal parameter surface. Exactly five ANSR parameters are exposed over ROS — the master topology, the training and validation data, an optional constraints file, and the results directory name. All remaining settings use the defaults of the core’s configuration, keeping the interface deliberately small; node-level behaviour is configured separately through ROS parameters (Section 1.5).

Guidelines compliance. Following D5.1: the node package is plain `ament_python` with the recommended `setup.py` layout; new interfaces live in the separate `physics_amm_msgs` package; the node file is a thin ROS shim over ROS-agnostic, independently testable modules (session staging, process supervision, result reading, session registry); a parameterised Python launcher and a `config.yaml` of default parameter values are provided; the license is Apache-2.0; and the repository targets quality Tier 2 with a continuous-integration workflow.

1.4 Interfaces

Table 1.1 summarises the public interface of the node. All names are relative to the node’s namespace (`/physics_amm` in the default launch configuration).

1.4.1 The `LearnModel` action

Listing 1.1 shows the complete action definition; the in-file comments are part of the specification. The goal carries the session id and the five ANSR parameters. Input paths are absolute paths on the host running the node; the node stages them into the session workspace before the run starts.

A goal is *rejected* (not queued) if the session does not exist, an input file is missing, or another learning run is currently active (Section 1.9); rejection is immediate and retryable. While the run

Listing 1.1: The LearnModel action definition (goal / result / feedback).

```

# Learn analytic models from data -- long-running (up to hours), cancellable.
# Goal carries the session id plus exactly the five agreed ANSR parameters;
# everything else takes its SRConfig default. Data is passed by file path
# (absolute, on the node host) and staged into the session workspace.
string session_id
string topology_path      # required; staged into <workspace>/topologies/
string train_data_path    # required; staged into <workspace>/data/
string valid_data_path    # required; staged into <workspace>/data/
string constraints_path    # "" = no constraints (-c omitted entirely)
string outfolder          # results dir name under the session workspace, e.g. "results"
---
int8 CODE_FINISHED=0      # ANSR exited 0 (ran its default budget to completion)
int8 CODE_CANCELLED=1     # client cancel or node watchdog; process group reaped
int8 CODE_FAILED=2        # nonzero exit or spawn failure; message carries stderr tail
int8 code
string message
AnalyticModel[] models    # final archive mirror (highest version); may be empty on failure
string results_path       # absolute <workspace>/<outfolder>/seed=1
---
float64 elapsed_seconds
string status              # "STARTING" (no mirror yet) / "RUNNING" / "FINALIZING"
uint32 dataset_version    # highest current_best_<v> observed (0 until first dump)
uint32 num_models         # individuals in the latest archive overview
float64 best_valid_loss    # NaN until the first mirror appears
float64 best_complexity    # complexity of the best individual (NaN until known)

```

executes, the node publishes feedback at a configurable period: elapsed time, a status string, the current dataset version, and a summary of the best model found so far (count, best validation loss, complexity).

The intended termination pattern is *cancel when satisfied*: because best-so-far models are available at any time through `get_model`, a client typically monitors the feedback (or polls the query service) and cancels the goal once the models are good enough. Cancellation is a first-class action operation, is always accepted, and cleanly terminates the whole computation (Section 1.6). A run that is neither cancelled nor stopped by the node-side wall-time watchdog ends when the core's internal budget is exhausted; the result then carries `CODE_FINISHED` together with the final models.

1.4.2 The AnalyticModel message

Learned models are returned as `AnalyticModel` messages (Table 1.2), one per model. The message is structurally parallel to `ModelDescription` in `coresense_msgs`, so a later migration to the shared `CoreSense` types is a field mapping.

1.4.3 Session and retrieval services

The four services are intentionally small; Table 1.3 lists their request and response fields.

Closing sessions. `end_session` with `force:=false` refuses to close a session whose learning run is still active — a safeguard against accidentally destroying an ongoing computation. With `force:=true`, the run is terminated first (through the same clean mechanism as an action cancellation) and the session workspace is then removed.

Querying models. `get_model` serves models from the core's best-so-far export — during a run, after it finishes, and after cancellation. By default it reads the *archive* (the non-dominated solutions,

Table 1.2: Fields of `physics_amm_msgs/msg/AnalyticModel`.

Field	Type	Meaning
<code>expression</code>	<code>string</code>	analytic formula in Python syntax
<code>simplified_expression</code>	<code>string</code>	symbolically simplified form; may be empty
<code>valid_loss</code>	<code>float64</code>	validation loss; NaN if not yet measured
<code>complexity</code>	<code>float64</code>	model complexity measure
<code>num_active_nodes</code>	<code>int32</code>	active nodes in the topology; <code>-1</code> if unknown
<code>rmse_constr</code>	<code>float64</code>	constraint-compliance error; NaN if no constraints
<code>dataset_version</code>	<code>uint32</code>	dataset version of the snapshot the model comes from
<code>population</code>	<code>string</code>	archive (non-dominated set) or <code>mainpop</code>
<code>txt_path</code>	<code>string</code>	path to the model's full text report
<code>m_path</code>	<code>string</code>	path to the MATLAB export of the formula

Table 1.3: Request and response fields of the session and retrieval services. All responses additionally carry `bool success` and `string message`.

Service	Request	Response (besides status)
<code>StartSession</code>	—	<code>session_id</code> (UUID)
<code>EndSession</code>	<code>session_id</code> ; <code>force</code>	—
<code>GetModel</code>	<code>session_id</code> ; <code>population</code> (empty = node default); <code>max_models</code> (0 = all)	<code>dataset_version</code> ; <code>AnalyticModel[] models</code>
<code>SetSessionData</code>	<code>session_id</code> ; <code>train_data_path</code> ; <code>valid_data_path</code> (empty = keep current)	—

i.e. the best accuracy–complexity trade-offs found so far); the live search population is available by requesting `population:=mainpop`. Because the export is refreshed at a minimum interval (5 s by default), a query may return a snapshot that is a few seconds behind the live search state; this is documented behaviour and is harmless for the best-so-far semantics of the call.

Updating data mid-run. `set_session_data` implements the *updating an existing model* mode. The node copies the new file's content over the staged training file — atomically, preserving the staged filename — so the core's data monitoring observes the update, reloads the data, and advances the dataset version. From that point on, models are re-evaluated on the new data and the export continues under the new version. The optional `valid_data_path` field updates the validation data in the same way.

1.5 Node parameters

Table 1.4 lists all node parameters with their defaults (from `config/params.yaml`, which also serves as the reference configuration for the launcher).

Three parameters matter in practice. `python_executable` selects the interpreter for the ANSR subprocess and should point at the virtual environment holding the scientific stack (Section 1.7); `ansr_source_dir` overrides the packaged copy of the core with a development checkout; and `workspace_root` chooses where session workspaces (staged data, logs, results) live — by default under `~/ros`, the conventional location for node-private state.

Table 1.4: Parameters of the `physics_amm` node.

Parameter	Default	Meaning
<code>workspace_root</code>	<code>~/.ros/physics_amm/sessions</code>	root directory for per-session workspaces
<code>ansr_source_dir</code>	<i>(installed copy)</i>	directory containing the ANSR core (<code>Main.py</code>)
<code>python_executable</code>	<i>(node's interpreter)</i>	interpreter used for the ANSR subprocess
<code>max_concurrent_runs</code>	1	learning runs executing at once (see Section 1.9)
<code>max_sessions</code>	8	open-session limit
<code>sigterm_grace_s</code>	10.0	grace period in the two-stage run termination
<code>feedback_period_s</code>	2.0	action feedback and cancellation-poll period
<code>max_run_seconds</code>	36000.0	node-side wall-time watchdog (0 disables it)
<code>default_population</code>	archive	population served by <code>get_model</code> when unspecified
<code>mirror_read_retries</code>	3	re-reads of an export snapshot being refreshed
<code>cleanup_workspace_on_end</code>	true	remove the session workspace on <code>end_session</code>

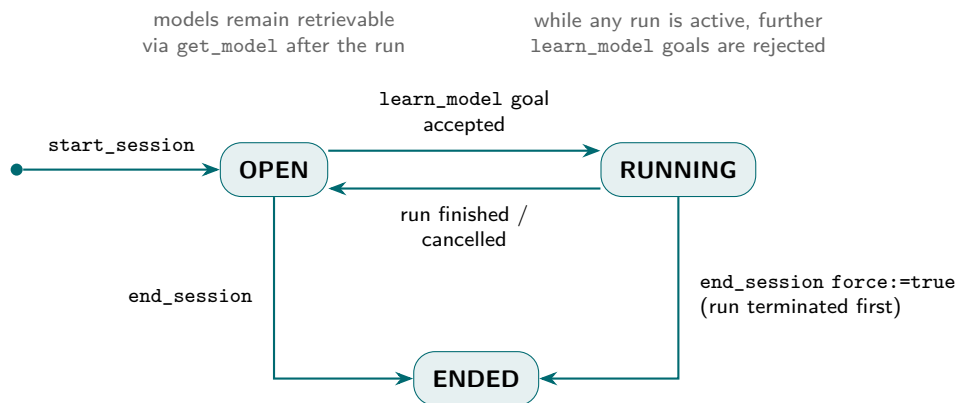


Figure 1.2: Session lifecycle. A session returns to `OPEN` after a run, keeping its results retrievable; closing it is an explicit, separate operation.

1.6 Implementation internals

The node file itself is a thin shim: all logic lives in four ROS-agnostic modules — `session` (workspace staging and command-line construction), `supervisor` (subprocess lifecycle), `results` (reading the best-so-far export), and `session_manager` (session registry and concurrency control) — which are unit-testable without a ROS installation. The node runs a multi-threaded executor with two callback groups, so retrieval services remain responsive while an action callback supervises a run.

1.6.1 Session lifecycle

Figure 1.2 shows the session state machine. A session is `OPEN` when created; it is `RUNNING` while a learning goal executes and returns to `OPEN` when the run ends — its workspace and results are retained, so `get_model` continues to serve the learned models. `end_session` moves the session to the terminal `ENDED` state and (by default) removes its workspace.

1.6.2 Workspace staging and core invocation

The workspace layout reproduces the core's directory conventions:

```

<session workspace>/
  data/           # training + validation CSV (staged copies)
  topologies/     # master topology file
  configs/        # constraints file (only when supplied)
  
```

Algorithm 1: Two-stage termination of a learning run's process group.

Input: process group g of the run, grace period T

```
1 send SIGTERM to every process in  $g$ ;  
2  $t_{\text{deadline}} \leftarrow \text{now} + T$ ;  
3 while  $\text{now} < t_{\text{deadline}}$  do  
4   if no process of  $g$  remains then return;  
5   wait briefly;  
6 send SIGKILL to every process in  $g$ ;  
7 wait until no process of  $g$  remains;
```

```
results/      # results directory targeted by the run  
logs/         # stdout/stderr of the ANSR subprocess
```

The core is invoked exactly as in a manual run: bare filenames for the inputs, the workspace as the working directory, and only the five exposed parameters on the command line. The subprocess environment pins the numerical libraries to one thread per worker process (`OMP_NUM_THREADS=1` and related variables), so the computational footprint of a run equals its worker pool and remains predictable on a shared robot computer. Staged copies also make the mid-run data update self-contained: the update rewrites the staged file atomically under the same name, which is precisely the event the core's data monitoring watches for.

1.6.3 Process supervision

A learning run is a small tree of processes: the core's main process, the Dask scheduler it hosts, and the worker pool, whose members are recycled by the core over the course of a run as part of its resource management. The supervisor therefore manages the run at the granularity of its *process group*, which is invariant across worker recycling: the subprocess is started as a new session leader, and all lifecycle operations address the group as a whole.

Termination — used by action cancellation, forced session closing, and the wall-time watchdog alike — follows the two-stage escalation of Algorithm 1: a cooperative SIGTERM to the whole group, a configurable grace period, then a definitive SIGKILL for any remainder. The supervisor verifies that no process of the group survives, so a cancelled run never leaves workers behind. This guarantee is exercised by a dedicated integration test (Section 1.8).

On the success path, the supervisor simply polls until the core exits and tolerates the core's unhurried finalisation: the final export is already complete on disk before the process ends, so the action result can be assembled reliably. Exit classification distinguishes a completed run, a cancelled run, and a failed run; in the failure case, the tail of the subprocess log is attached to the action result's message for diagnosis. As a safety net for unattended deployments, the node-side wall-time watchdog (`max_run_seconds`) terminates a run through the same mechanism and reports it as cancelled.

1.6.4 Retrieving models from the export directory

Within a session's results directory, the core exports its best-so-far models per population and dataset version:

```
<results>/seed=1/  
  archiveIndividuals/current_best_<v>/      # non-dominated solutions  
  mainPopulationIndividuals/current_best_<v>/ # live search population
```

Each snapshot directory holds one text report and one MATLAB export per model, plus an overview file summarising the population. The reader component selects the *highest* version present — after a mid-run data update, a new version appears and earlier versions remain as snapshots of the previous dataset — and parses the overview together with the per-model reports into `AnalyticModel` messages, preserving the overview’s ordering.

Two timing properties follow from the continuous-export design and are handled by the reader. First, a snapshot that is being refreshed at the moment of a query may be momentarily incomplete; reads are therefore retried briefly and degrade gracefully to an empty result with an explanatory message rather than failing. Second, the export interval bounds the staleness of any answer (5 s by default), and immediately after a data update the new version’s snapshot fills up as models are re-evaluated — clients polling right after `set_session_data` should expect the new version to appear within a few export intervals.

1.7 Building and running

The module is built like any ROS 2 workspace, with one addition: the ANSR scientific stack (PyTorch, Dask, SymPy) lives in a Python virtual environment created with system site-packages visible, so that both the ROS client libraries and the scientific stack are importable from one interpreter.

Listing 1.2: Build recipe (Ubuntu 24.04, ROS 2 Jazzy).

```
sudo apt install ros-jazzy-ros-base python3.12-venv \
    python3-colcon-common-extensions

# venv on the same interpreter ROS Jazzy uses (Python 3.12); the venv is
# used via absolute paths and never needs to be activated
python3.12 -m venv --system-site-packages ~/venvs/ansr

REPO=$(pwd) # run from the repository root: absolute path for the steps below
~/venvs/ansr/bin/pip install -r $REPO/source_code/requirements.txt

source /opt/ros/jazzy/setup.bash

mkdir -p ~/ros2_ws/src
ln -sfn $REPO/ros2 ~/ros2_ws/src/physics_amm_ros
cd ~/ros2_ws && colcon build --symlink-install
source install/setup.bash # repeat in every new terminal
```

The `python_executable` launch argument (first line of Listing 1.3) points the ANSR subprocess at this virtual environment’s interpreter, so that the subprocess runs with the scientific stack independently of which interpreter serves the node itself; the node reports the effective interpreter in its startup log. Listing 1.3 walks through a complete session with the standard `ros2` command-line tools; the same calls are available programmatically from any ROS 2 client library.

Listing 1.3: A complete session from the command line.

```
ros2 launch physics_amm physics_amm.launch.py \
    python_executable:=~/venvs/ansr/bin/python

# 1. open a session
ros2 service call /physics_amm/start_session \
    physics_amm_msgs/srv/StartSession

# 2. training a new model: send a learning goal (absolute paths)
ros2 action send_goal /physics_amm/learn_model \
    physics_amm_msgs/action/LearnModel \
    "{session_id: '<uuid>', topology_path: '<abs>/topology.txt',
     train_data_path: '<abs>/train.csv',
```

```

    valid_data_path: '<abs>/valid.csv',
    constraints_path: '', outfolder: results}" --feedback

# 3. querying: best-so-far models, any time during or after the run
ros2 service call /physics_amm/get_model \
  physics_amm_msgs/srv/GetModel \
  "{session_id: '<uuid>', population: '', max_models: 3}"

# 4. updating: replace the training data mid-run
ros2 service call /physics_amm/set_session_data \
  physics_amm_msgs/srv/SetSessionData \
  "{session_id: '<uuid>', train_data_path: '<abs>/new_train.csv',
    valid_data_path: ''}"

# 5. cancel when satisfied (Ctrl-C on send_goal, or the action API),
# then close the session
ros2 service call /physics_amm/end_session \
  physics_amm_msgs/srv/EndSession \
  "{session_id: '<uuid>', force: true}"

```

Steps 2–4 are exactly the three operation modes announced in D3.7: training, querying, and updating are one action goal and two service calls. Data files are headerless comma-separated CSV with the target variable in the last column.

1.8 Verification

ANSR is an asynchronous evolutionary method: the order in which tuned candidates return from the worker pool depends on scheduling, so two runs with identical inputs legitimately produce different (equally valid) expressions. Verification of the ROS layer therefore does not compare expressions or loss values against a reference run. Instead, the test suite asserts the *integration contract* — that the node invokes and observes the core exactly as a manual invocation would — and *invariants* that must hold for any run. This is where integration defects would actually appear: in the command line, the workspace layout, the results path, or process lifecycle management, all of which are deterministic and precisely testable.

Table 1.5 shows the six test layers.

The layers that execute the core share a set of bundled smoke fixtures: 60 training and 40 validation samples with inputs drawn uniformly from $[-2, 2]$ and *noise-free* targets $y = x_0^2 + x_1$ (plus an alternative dataset with $y = x_0^2 + 2x_1$ for the mid-run data update of layer L5), paired with a minimal master topology that can represent both targets exactly. Short runs therefore converge to near-zero validation loss, which keeps the “did it learn anything?” assertions meaningful within continuous-integration time budgets — and any significant residual implicates the integration, not the data.

Layers L1 and L2 are fast, deterministic unit tests that run on every commit; they pin the exact command line (only the five agreed parameters, bare filenames) and the workspace layout. Layer L4 compares a run driven through ROS against the direct-invocation baseline of L3 *structurally* — same directory tree, same artifact kinds, finite losses — never by values. Layer L5 exercises the mid-run data update end-to-end and verifies that queries are answered while the action is executing. Layer L6 verifies the process-group termination guarantee of Section 1.6.3: after a cancellation, no process of the run’s group remains within seconds. In addition, the packages carry unit tests for the ROS-agnostic modules and a launch test that brings the node up through its launcher, meeting the D5.1 requirement of testing both the module API and the node; together the suites reach the coverage level of quality Tier 2.

Table 1.5: Test layers of the ROS bundle.

#	Layer	ROS	Speed	What it proves
L1	command-line contract	no	ms	the node invokes ANSR exactly as a human would
L2	workspace staging	no	ms	the staged layout matches the core's conventions
L3	reference smoke	no	~1 min	baseline output structure of a direct run
L4	ROS smoke	yes	~1 min	the action reproduces the L3 structure
L5	data update mid-run	yes	~2 min	dataset version advances; queries answer mid-run
L6	cancellation	yes	~30 s	termination leaves no process of the run behind

1.9 Operational notes and outlook

One run at a time. By default the node executes one learning run at a time (parameter `max_concurrent_runs`, default 1); goals arriving while a run is active are rejected immediately with an explanatory message, and the client retries once the slot is free. Serialising runs keeps the CPU footprint of the module equal to a single worker pool — a deliberate choice for shared robot computers, where the module coexists with perception and control. The cap is a parameter, so a workstation deployment with ample cores may raise it, giving each session its own isolated workspace and results directory.

Termination policy. Learning is an anytime computation: the best-so-far models are continuously available, so clients decide when the result is good enough and cancel. The node-side watchdog (`max_run_seconds`) bounds unattended runs; its default equals the core's own time budget, so it acts purely as a safety net.

Outlook. Three integration steps are foreseen. First, the `AnalyticModel` message was designed field-compatible with `ModelDescription` in `coresense_msgs`, so migrating to the shared `CoreSense` types once the consortium consolidates interfaces is a mechanical mapping. Second, the module ships its own launcher and default configuration, ready to be composed into a system-level bringup (`coresense_bringup`). Third, following the D5.1 release process, the packages will be released with `bloom` from the `jazzy-devel` branch once the interface has settled in consortium use.

Bibliography

- [1] Jiří Kubalík and Robert Babuška. D3.7 Physics-aware modeling module (CS-069 v1.0). Technical report, CoreSense Horizon Europe Project #101070254, 2025.
- [2] CoreSense Consortium. D5.1 CoreSense/ROS Development Guidelines (CS-019 v2.0). Technical report, CoreSense Horizon Europe Project #101070254, 2025.
- [3] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.
- [4] Jiří Kubalík, Erik Derner, and Robert Babuška. Toward physically plausible data-driven models: A novel neural network approach to symbolic regression. *IEEE Access*, 11:61481–61501, 2023.
- [5] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference (SciPy)*, pages 126–132, 2015.
- [6] CoreSense Consortium. coresense_vampire: a ROS2 wrapper around the Vampire theorem prover. https://github.com/CoreSenseEU/coresense_vampire, 2024.